24ME511 PRODUCT DEVELOPMENT LAB – 2

SENTINEL: AI-POWERED CITIZEN INCIDENT REPORTING AND INTELLIGENT EMERGENCY DISPATCH SYSTEM

A PROJECT REPORT

Submitted by

LATHESHKUMAR S	**(111724102105)**
MADESH S	**(111724102108)**
JEFFRIN A	**(111724102083)**
KAMALESH BALAJI D N	**(111724102089)**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

R.M.K. ENGINEERING COLLEGE

(An Autonomous Institution)

R.S.M. Nagar, Kavaraipettai-601 206



OCTOBER 2026

R.M.K. ENGINEERING COLLEGE

(An Autonomous Institution)
R.S.M. Nagar, Kavaraipettai-601 206

BONAFIDE CERTIFICATE

Certified that this Product Development Lab – II report **"SENTINEL: AI-POWERED CITIZEN INCIDENT REPORTING AND INTELLIGENT EMERGENCY DISPATCH SYSTEM"** is the Bonafide work of **Latheshkumar S (111724102105), Madesh S (111724102108), Jeffrin A (111724102083), Kamalesh Balaji D N (111724102089)** who carried out the **24ME511 Product Development Lab – 2** work under my supervision.

SIGNATURE

Dr. T. Sethukarasi M.E., M.S. Ph.D.,
Professor and Head
Department of Computer Science and Engineering
R.M.K. Engineering College,
R.S.M. Nagar, Kavaraipettai,
Tiruvallur District– 601206.

SIGNATURE

Ms. Sumitha T
Supervisor
Assistant Professor
Department of Computer Science and Engineering
R.M.K. Engineering College,
Tiruvallur District–601206.

Submitted for the Product Development Lab – 2 Viva-Voce held on …… at R.M.K. Engineering College, Kavaraipettai, Tiruvallur District– 601206.

INTERNAL EXAMINER

ACKNOWLEDGEMENT

We earnestly portray our sincere gratitude and regard to our beloved **Chairman Shri. R. S. Munirathinam, our Vice Chairman, Shri. R. M. Kishore** and our **Director, Shri. R. Jyothi Naidu,** for the interest and affection shown towards us throughout the course.

We convey our sincere thanks to our **Principal, Dr. K. A. Mohamed Junaid,** for being the source of inspiration in this college.

We reveal our sincere thanks to our **Professor and Head of the Department, Computer Science and Engineering, Dr. T. Sethukarasi,** for her commendable support and encouragement for the completion of our project.

We would like to express our sincere gratitude to our **Product Development Lab Incharge and Supervisor Ms. Sumitha T, Assistant Professor,** for the valuable suggestions and guidance towards the successful completion of this project in a global manner.

We take this opportunity to extend our thanks to all faculty members of the Department of Computer Science and Engineering, Parents and friends for all that they meant to us during the crucial times of the completion of our project.

ABSTRACT

Emergency response in Indian cities still depends largely on voice calls to helplines, where a caller has to describe an unfolding situation verbally, the operator has to type it in, and the location is often vague or wrong. When a single event such as a transformer fire or a road accident is witnessed by many people, the same incident is reported several times and each call competes for the attention of the same few dispatchers. The proposed system, “Sentinel: AI-Powered Citizen Incident Reporting and Intelligent Emergency Dispatch System”, addresses these problems with a progressive web application through which any citizen can report a hazard in seconds by choosing a category, describing it in plain language, attaching a photo, video or voice note, and sharing the GPS position of the phone. Every report is passed through an artificial intelligence pipeline that detects objects in images and video keyframes using YOLOv8, verifies image metadata with EXIF analysis, transcribes voice notes with Whisper, and uses a locally hosted large language model to extract a structured summary, hazard type and severity from free text. Sentence embeddings combined with a Union-Find (disjoint set) algorithm merge duplicate reports that are within 200 metres, two hours and semantically similar, and a weighted priority score orders the queue. A dispatcher dashboard shows all incidents live on a map of Chennai, receives updates over WebSockets, and recommends the nearest available responders using a k-d tree spatial index. The system turns scattered, duplicated and unstructured citizen reports into a single prioritised and verified incident queue, reducing the time between a citizen noticing a hazard and a responder being dispatched.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	iv
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	viii
1	**INTRODUCTION**	01
2	**LITERATURE SURVEY**	04
3	**PROBLEM DEFINITION**	08
	3.1 PROBLEM STATEMENT	08
	3.2 EXISTING SYSTEM	08
	3.3 PROPOSED SYSTEM	09
4	**SYSTEM STUDY**	11
	4.1 FEASIBILITY STUDY	11
	4.1.1 Legal Feasibility	11
	4.1.2 Operational Feasibility	11
	4.1.3 Economic Feasibility	11
	4.1.4 Technical Feasibility	12
5	**SYSTEM REQUIREMENTS SPECIFICATION**	13
	5.1 SOFTWARE REQUIREMENTS	13
	5.2 HARDWARE REQUIREMENTS	13
	5.3 SOFTWARE OVERVIEW	13
6	**SYSTEM DESIGN**	15
	6.1 SYSTEM ARCHITECTURE	15
	6.2 DATA FLOW DIAGRAM	15
	6.3 UML DIAGRAM	16
	6.3.1 Use Case Diagram	16
	6.3.2 Class Diagram	17
	6.3.3 Activity Diagram	18
7	**SYSTEM IMPLEMENTATION**	20
	7.1 MODULES	20
	7.2 MODULE DESCRIPTION	20
	7.2.1 Citizen Reporting Module	20
	7.2.2 Media Storage and Validation Module	20
	7.2.3 Computer Vision Module	21
	7.2.4 Speech and Language Processing Module	21
	7.2.5 Duplicate Detection Module	22
	7.2.6 Priority Scoring Module	22
	7.2.7 Dispatcher Dashboard and Real-time Module	22
	7.2.8 Responder Dispatch and Authentication Module	23
8	**SAMPLE CODE**	24
9	**SYSTEM TESTING AND MAINTENANCE**	28
	9.1 UNIT TESTING	28

9.2 FUNCTIONAL TESTING 28
9.3 INTEGRATION TESTING 29
9.4 BLACK BOX TESTING 29
9.5 WHITE BOX TESTING 29
10 SCREENSHOTS 30
11 CONCLUSION & FUTURE ENHANCEMENT 34
11.1 CONCLUSION 34
11.2 FUTURE ENHANCEMENT 34
REFERENCES 36

LIST OF FIGURES

Figure No	Name	Page No
1.1	Public Safety Domain Overview	03
5.1	Sentinel System Components	14
6.1	System Architecture	15
6.2	Data Flow Diagram	16
6.3.1	Use Case Diagram	17
6.3.2	Class Diagram	18
6.3.3	Activity Diagram	19
10.1	Citizen Report Form	30
10.2	Report with Photo Evidence and GPS	30
10.3	Citizen Track Status	31
10.4	Dispatcher Login	31
10.5	Dispatcher Dashboard	32
10.6	AI Incident Analysis	32
10.7	Nearest Responder Dispatch	33
10.8	Analytics Dashboard	33

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
CNN	Convolutional Neural Network
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
DSU	Disjoint Set Union
EXIF	Exchangeable Image File Format
GGUF	GPT-Generated Unified Format
GPS	Global Positioning System
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NLP	Natural Language Processing
ORM	Object Relational Mapping
PWA	Progressive Web Application
REST	Representational State Transfer
SQL	Structured Query Language
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

Overview

Every day, citizens in a large city such as Chennai witness road accidents, fires, waterlogged roads, fallen electric wires, collapsing structures and medical emergencies. The quickest way for them to raise an alarm today is to call a helpline such as 100, 101, 108 or 112. A voice call is a poor channel for this job: the caller is often panicking, the description is incomplete, the location is given as a landmark that the operator has to interpret, and there is no way to share a photograph that would immediately show how serious the situation is. When a highly visible event happens at a busy junction, dozens of people may call about the same incident, and each of those calls occupies an operator while genuinely new emergencies wait in the queue.

The proposed system, “Sentinel: AI-Powered Citizen Incident Reporting and Intelligent Emergency Dispatch System”, replaces this unstructured process with a digital one. A citizen opens a lightweight progressive web application on any phone, selects the type of hazard, writes or speaks a short description, attaches a photo, video or voice note and shares the exact GPS location. The report is stored immediately and the citizen receives a tracking ID. In the background, an artificial intelligence pipeline analyses every piece of evidence: YOLOv8 detects objects in images and video keyframes, EXIF metadata is checked against the reported location and time, Whisper converts voice notes to text, and a locally hosted large language model extracts a structured summary, hazard type and severity. Sentence embeddings and a Union-Find algorithm merge duplicate reports of the same event, and a weighted priority score places the most urgent incidents at the top of the dispatcher’s queue. Dispatchers see every incident live on a map, and the system recommends the nearest available responders using a k-d tree spatial index.

Domain

The proposed system belongs to the domain of Public Safety and Emergency Management, specifically to AI-assisted Computer Aided Dispatch. Public safety systems aim to detect, assess and respond to threats to life and property as quickly as possible. As cities grow and smartphones become universal, citizens themselves have become the largest network of sensors available to emergency services, and the challenge has shifted

from collecting reports to making sense of them quickly. The proposed system integrates multiple sub-domains, including:

Crowdsourced Incident Reporting: Citizens act as real-time sensors. A mobile-first web application lets anyone submit a report with text, media and location without installing a native app or creating an account, which keeps the barrier to reporting as low as possible.

Computer Vision: Object detection with YOLOv8 identifies vehicles, people, buses, trucks and other objects in photographs and in keyframes sampled from videos. The detections provide independent evidence of what the citizen has described and contribute to the severity estimate.

Natural Language Processing: A large language model running locally through llama.cpp converts free text into a structured record containing the hazard type, location name, number of people affected, severity and a one-line summary. Whisper speech recognition converts voice notes into text so that people who cannot type during an emergency can still report.

Geospatial Computing: GPS coordinates, the Haversine distance formula and a k-d tree are used to find duplicate reports close to each other and to locate the nearest available responder.

Decision Support and Real-time Systems: Duplicate clustering, priority scoring and live WebSocket updates turn a stream of individual reports into a single ordered queue that a dispatcher can act on.

Advantages

The public safety domain combined with artificial intelligence offers several advantages that make it a promising area for innovation:

- **Faster Reporting** – A citizen can file a complete report with evidence and an exact location in under a minute, without waiting for a helpline call to be answered.
- **Accurate Location** – GPS coordinates replace vague verbal descriptions, so responders reach the right spot the first time.
- **Visual Evidence** – Photos, videos and voice notes give dispatchers far more information than a phone call and allow AI models to verify what is being reported.

- **Duplicate Elimination** – Many reports of the same event are merged into one cluster, so dispatchers deal with each emergency once and the number of corroborating reports increases its priority.
- **Objective Prioritisation** – A transparent, weighted score based on severity, corroboration, waiting time and category orders the queue consistently instead of relying on who called first.
- **Faster Dispatch** – The nearest available responder is suggested automatically, reducing the time between a report and a unit being sent.
- **Accountability** – Every status change from reported to closed is time-stamped, so response-time metrics can be measured and improved.

Applications

The proposed system has applications across many areas of public safety and civic administration:

- **City Emergency Control Rooms** – Police, fire and ambulance control rooms can use the dashboard as a single view of every citizen-reported incident in the city.
- **Disaster and Flood Management** – During monsoon flooding in Chennai, thousands of waterlogging reports can be clustered by location so that pumps and rescue teams are sent where they are needed most.
- **Traffic Police** – Road accidents, overturned vehicles and blocked junctions can be reported with photos and located precisely on the map.
- **Electricity Boards** – Fallen live wires and transformer fires can be routed to electrical hazard teams.
- **Municipal Corporations** – Potholes, collapsing structures and drainage overflow can be logged and tracked through to resolution.
- **Campuses and Industrial Parks** – Large private campuses can run their own instance for internal safety reporting.

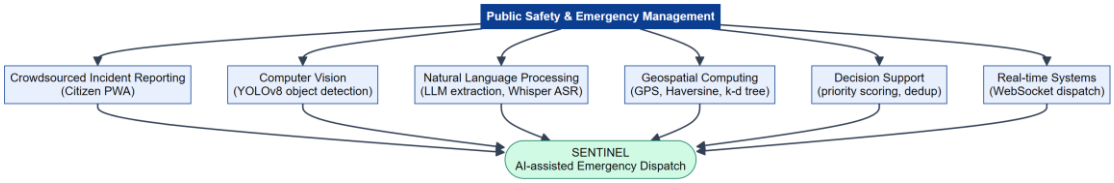


Fig 1.1 Public Safety Domain Overview

CHAPTER 2

LITERATURE SURVEY

2.1 Paper Title: Earthquake Shakes Twitter Users: Real-time Event Detection by Social Sensors

Author : Takeshi Sakaki, Makoto Okazaki, Yutaka Matsuo (WWW 2010)

This paper treats every Twitter user as a sensor and shows that real-world events such as earthquakes can be detected from the messages people post. The authors train a support vector machine on keywords, the number of words and the context of each tweet to decide whether it reports an actual earthquake, and then apply probabilistic models including Kalman and particle filters to estimate the location and centre of the event from the positions of the reporting users. Their system detected a large share of earthquakes in Japan and sent notifications faster than the official broadcast. The work established that crowdsourced reports can be fast and reliable when aggregated. However, it relies on public social media posts that are short, noisy and often lack precise coordinates, and it only detects that an event happened; it does not support dispatching responders. This motivates a dedicated reporting channel that captures precise GPS and media evidence.

2.2 Paper Title: Processing Social Media Messages in Mass Emergency: A Survey

Author : Muhammad Imran, Carlos Castillo, Fernando Diaz, Sarah Vieweg (ACM Computing Surveys, 2015)

This survey reviews computational methods for handling the very large volume of messages that are posted during disasters. It covers event detection, classification of messages into information types, extraction of actionable information, summarisation and the problem of credibility and misinformation. The authors highlight that during emergencies the main problem is not a lack of information but information overload, and that duplicate and redundant messages make up a large part of the stream. They also note that most research systems stop at analysis and are rarely integrated with the operational tools used by responders. The survey motivates the de-duplication and structured extraction stages of the proposed system and its direct integration with a dispatcher workflow.

2.3 Paper Title: Crowdsourcing Geographic Information for Disaster Response: A Research Frontier

Author : Michael F. Goodchild, J. Alan Glennon (International Journal of Digital Earth, 2010)

The authors study volunteered geographic information, where ordinary citizens contribute location-based reports, using the example of the 2007–2009 wildfires in Santa Barbara. They show that volunteer maps were often available hours before official information and were highly valuable to residents and agencies. At the same time they discuss the risks of inaccurate or false reports and the absence of quality control. The paper concludes that crowdsourced geographic data is essential for disaster response but must be combined with verification mechanisms. The proposed system follows this direction by collecting precise GPS coordinates and verifying them, where possible, against the EXIF metadata embedded in photographs.

2.4 Paper Title: CrisisMMD: Multimodal Twitter Datasets from Natural Disasters

Author : Firoj Alam, Ferda Ofli, Muhammad Imran (ICWSM 2018)

This paper introduces a large annotated dataset of tweets containing both text and images from seven natural disasters, including earthquakes, hurricanes, floods and wildfires. Each item is labelled for whether it is informative, which humanitarian category it belongs to and how severe the damage shown in the image is. The dataset demonstrated that images frequently carry information that is not present in the text, such as the extent of damage, and that combining both modalities gives a better picture of an incident. Its limitation is that it is an offline research resource built from social media and does not include a reporting or response pipeline. It supports the design decision to accept photos and videos alongside the text of every report in the proposed system.

2.5 Paper Title: Analysis of Social Media Data using Multimodal Deep Learning for Disaster Response

Author : Ferda Ofli, Firoj Alam, Muhammad Imran (ISCRAM 2020)

Building on CrisisMMD, this study trains convolutional neural networks on images and text jointly to classify disaster-related posts as informative or not and to assign humanitarian categories. The multimodal models outperformed models that used only text or only images, confirming that the two sources complement each other. The authors note that deep models require substantial labelled data and computational resources, and that

the work concentrates on classification rather than on the operational steps that follow it. The proposed system adopts the multimodal principle but uses pretrained general-purpose models (YOLOv8 for images and a local large language model for text) so that it works without training a custom dataset.

2.6 Paper Title: You Only Look Once: Unified, Real-Time Object Detection

Author : Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi (CVPR 2016)

This paper introduced YOLO, which reframes object detection as a single regression problem from image pixels to bounding boxes and class probabilities. A single convolutional network looks at the whole image once and predicts all objects simultaneously, which made it dramatically faster than earlier region-proposal methods while remaining accurate enough for practical use. Later versions, including YOLOv8 by Ultralytics used in this project, improved accuracy and added small, efficient models that run on ordinary laptops. The limitation of the original model was lower accuracy on small objects. For an incident reporting system, the speed of YOLO means every uploaded image and sampled video keyframe can be analysed within seconds of submission, which is why it was chosen for the vision stage of the proposed pipeline.

2.7 Paper Title: Robust Speech Recognition via Large-Scale Weak Supervision

Author : Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, Ilya Sutskever (ICML 2023)

This paper presents Whisper, a speech recognition model trained on 680,000 hours of multilingual audio collected from the web. Because of the scale and diversity of its training data, Whisper generalises well to new accents, background noise and languages without any fine-tuning, approaching human-level robustness on many benchmarks. Its main drawbacks are the computational cost of the larger models and occasional hallucinated text on silent or very noisy audio. In the proposed system, Whisper allows a citizen to record a voice note instead of typing, which is important when the reporter is driving, injured or in a hurry, and the transcript is appended to the report description for further analysis.

2.8 Paper Title: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

Author : Nils Reimers, Iryna Gurevych (EMNLP 2019)

The authors modify the BERT language model with a siamese network structure so that it produces a fixed-size vector for each sentence, such that sentences with similar meanings have a high cosine similarity. This reduced the time to find the most similar pair in a collection of 10,000 sentences from about 65 hours with plain BERT to about 5 seconds, while keeping BERT's accuracy. The limitation is that the embeddings capture meaning only as well as the training data allows, so synonyms with no shared vocabulary can still score low. The proposed system uses the all-MiniLM-L6-v2 sentence-transformer model from this family to measure whether two incident descriptions refer to the same event, with a threshold calibrated on hand-written duplicate and non-duplicate pairs.

2.9 Paper Title: Multidimensional Binary Search Trees Used for Associative Searching

Author : Jon Louis Bentley (Communications of the ACM, 1975)

This paper introduced the k-d tree, a binary search tree in which each level splits the data along one of k dimensions in turn. It supports nearest-neighbour and range queries in logarithmic average time instead of comparing the query against every stored point. The structure is simple to build and works best in low dimensions, which makes it ideal for two-dimensional geographic data. Its performance degrades in very high dimensions. In the proposed system the responder positions form a small two-dimensional set, so a k-d tree with a latitude-aware distance bound is used to return the nearest available responders for any incident.

2.10 Paper Title: Emergency Medical Services and Beyond: Addressing New Challenges through a Wide Literature Review

Author : Roberto Aringhieri, Maria Elena Bruni, Sara Khodaparasti, J. Theresia van Essen (Computers & Operations Research, 2017)

This review surveys operations research approaches to emergency medical services, covering the location of ambulance bases, dispatching and relocation policies, and the integration of emergency services with hospitals and other care providers. The authors show that dispatching decisions such as sending the closest available unit have a large effect on response times and survival rates, and identify the lack of real-time, high-quality incident information as a key obstacle to better decisions. The proposed system responds to this by supplying dispatchers with verified, prioritised and geolocated incidents together with a nearest-responder recommendation.

CHAPTER 3

PROBLEM DEFINITION

3.1 PROBLEM STATEMENT

In most Indian cities, emergencies are reported to authorities mainly through voice calls to helplines. These calls carry only what the caller can say: there is no photograph or video of the scene, the location is described verbally and often inaccurately, and the operator must manually type and classify every detail. When a visible incident occurs at a busy place, many people report it separately and each report is treated as a new call, which overloads operators and delays genuinely new emergencies. There is no automatic way to judge how severe an incident is, to verify whether a report is genuine, to merge duplicate reports or to decide which incident should be handled first. Dispatchers must also work out manually which responder is closest. As a result, valuable minutes are lost between the moment a citizen notices a hazard and the moment a responder is sent. Therefore, there is a need for an intelligent system that lets citizens report incidents with evidence and exact location, automatically analyses and verifies those reports, merges duplicates, prioritises them, and helps dispatchers send the nearest responder.

3.2 EXISTING SYSTEM

The existing system for emergency reporting consists of helpline numbers (100 for police, 101 for fire, 108 for ambulance and the integrated 112), complaint portals of municipal corporations, and informal reporting through social media. Control room operators receive calls, record the details in a computer aided dispatch or ticketing system and assign a unit by radio or phone.

Common Features of Existing Systems:

- Citizens report emergencies by calling a helpline and describing the situation verbally to an operator.
- Operators manually record the caller’s description, the approximate location and the type of emergency in a ticketing or dispatch system.
- Some states provide mobile apps or web portals, but these usually act as simple complaint forms without any automated analysis of the content.

- Assignment of responders is done manually by the operator based on their knowledge of the area.

Disadvantages of the Existing System:

Despite their importance, existing emergency reporting systems face several significant limitations:

No Visual Evidence: A voice call cannot carry a photo or video, so the dispatcher cannot see how serious the situation is. Severity has to be guessed from the caller's words, which are often affected by panic.

Inaccurate Location: Callers describe locations using landmarks or approximate area names. Operators must interpret these descriptions, and responders may waste time searching for the exact spot.

Duplicate Reports: A single highly visible incident can generate many separate calls. Each one is handled as a new case, which wastes operator time and makes the queue look longer than it is.

Manual Prioritisation: There is no objective, consistent way to decide which incident to handle first. Calls are generally processed in the order received rather than by severity.

Slow Dispatch: Finding the nearest available responder depends on the operator's memory and radio communication, which takes time and does not always produce the best choice.

No Tracking for Citizens: After calling, the citizen usually has no way to know whether help has been sent, which leads to repeated calls that further overload the system.

3.3 PROPOSED SYSTEM

The proposed system, Sentinel, is an AI-powered citizen incident reporting and dispatch platform. Citizens report hazards through a progressive web application with text, media and GPS location. Each report is analysed by an AI pipeline that combines computer vision, metadata verification, speech recognition and a large language model, then de-duplicated and prioritised before it appears on a live dispatcher dashboard with nearest-responder recommendations.

Features of the Proposed System:

- **Multimodal Reporting** – Citizens choose one of eight hazard categories, describe the incident, and attach photos, videos or voice notes along with GPS coordinates captured from the phone.
- **Instant Acknowledgement and Tracking** – Every report returns a tracking ID immediately, and the citizen can follow its progress through seven lifecycle stages from Reported to Closed.
- **AI Media Analysis** – YOLOv8 detects objects in images and in three keyframes sampled from each video; EXIF metadata is checked against the reported location and time; Whisper transcribes audio.
- **Structured Extraction** – A local large language model (Qwen3.5 served by llama.cpp) converts free text into a structured record with hazard type, location, affected people, severity and a summary.
- **Duplicate Clustering** – Sentence embeddings and a Union-Find algorithm merge reports within 200 metres and two hours whose descriptions are semantically similar.
- **Priority Queue** – A weighted score of severity (40%), corroboration (30%), waiting time (20%) and category (10%) orders incidents for the dispatcher.
- **Live Dashboard** – Dispatchers see all incidents on an interactive map of Chennai with a heatmap overlay, receive real-time updates over WebSockets and can filter the queue by category.
- **Nearest Responder Dispatch** – A k-d tree returns the closest available responders, and the dispatcher can assign one with a single click.
- **Honest Degradation** – If any AI model is unavailable, the system records that fact instead of inventing results, so dispatchers always know what was actually analysed.

Advantages of the Proposed System:

- Reduces the time taken to report an incident and to dispatch a responder.
- Provides accurate GPS location and visual evidence for every report.
- Eliminates duplicate handling by merging reports of the same event.
- Orders incidents objectively so the most urgent are handled first.
- Keeps citizens informed, reducing repeated follow-up calls.
- Runs the language model locally, so citizen reports are not sent to external cloud AI services.

CHAPTER 4

SYSTEM STUDY

4.1 FEASIBILITY STUDY

A feasibility study evaluates whether the proposed system can be developed and operated successfully. It examines the system from legal, operational, economic and technical points of view to make sure that the project is practical, affordable and beneficial before full implementation.

4.1.1 Legal Feasibility

The system is built entirely with open-source software released under permissive licences, including React, FastAPI, PostgreSQL, Redis, Leaflet and the llama.cpp runtime. Map data comes from OpenStreetMap under the Open Database Licence with the required attribution displayed on the map. The YOLOv8 model is available under the AGPL-3.0 licence, which is suitable for an academic and public-interest project. Citizen data is handled with privacy in mind: reporting does not require an account, the contact phone number is optional, SQL query logging is disabled so that personal details do not appear in logs, and the language model runs locally so report contents are not shared with third-party AI providers. This approach is consistent with the principles of India's Digital Personal Data Protection Act, 2023.

4.1.2 Operational Feasibility

The citizen interface is a simple form that works in any modern mobile browser and can be installed to the home screen as a progressive web application, so no app store download is required. Large category buttons, a single "Use my location" action and optional voice notes keep the reporting process quick even under stress. The dispatcher dashboard presents a familiar map, a sorted queue and one-click dispatch, so control room staff need very little training. Because the AI pipeline runs in the background, citizens receive their tracking ID immediately and are never kept waiting for analysis to complete.

4.1.3 Economic Feasibility

All software components are free and open source, so there are no licence fees. The AI models run on a single laptop-class machine with a consumer GPU; no paid cloud AI API is required, which avoids per-request costs that would grow with the number of reports.

The system can be deployed on one server with Docker Compose and scaled later by adding Celery workers. Compared with the cost of delayed emergency response, including loss of life and property, the investment is small and the benefits are significant.

4.1.4 Technical Feasibility

The project uses mature and well-documented technologies. FastAPI provides a high-performance asynchronous web server, SQLAlchemy handles database access, and Celery with Redis distributes AI processing to background workers. Pretrained models remove the need to collect and label training data: YOLOv8n is a 6 MB model that runs in real time, all-MiniLM-L6-v2 produces 384-dimensional sentence embeddings on a CPU, and a quantised Qwen3.5 model in GGUF format runs on an RTX 4060 laptop GPU through llama.cpp. Every AI component falls back gracefully when its model is missing, and the whole system can also run in a standalone mode with SQLite and no Docker, which was verified during development. The prototype is therefore technically feasible.

CHAPTER 5

SYSTEM REQUIREMENTS SPECIFICATION

5.1 SOFTWARE REQUIREMENTS

Component	Specification
Operating System	Windows 10/11, Ubuntu 22.04 or any OS with Docker
Backend Language	Python 3.10 or above
Web Framework	FastAPI with Uvicorn ASGI server
Frontend	React 18, Vite, Tailwind CSS v4, Leaflet
Database	PostgreSQL 16 with PostGIS (SQLite for standalone mode)
Task Queue	Celery with Redis broker
AI Models	YOLOv8n, OpenAI Whisper, all-MiniLM-L6-v2, Qwen3.5 (GGUF)
LLM Runtime	llama.cpp llama-server (CUDA build)
Browser	Google Chrome, Microsoft Edge or Firefox (latest)
Tools	Visual Studio Code, Git, Docker Desktop, Node.js 20

5.2 HARDWARE REQUIREMENTS

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or above
RAM	16 GB (8 GB minimum without the local LLM)
GPU	NVIDIA GPU with 6 GB+ VRAM (RTX 4060 used) for the LLM
Storage	20 GB free space for models, database and media
Client Device	Any smartphone with a browser, camera and GPS
Network	Internet or local network connection

5.3 SOFTWARE OVERVIEW

React and Vite: React is a JavaScript library for building user interfaces from reusable components. The citizen form, tracking screen, dashboard and analytics pages are React components, and Vite builds it as a progressive web app that can be installed on a phone's home screen.

Leaflet and OpenStreetMap: Leaflet is a lightweight open-source library for interactive maps. It renders OpenStreetMap tiles of Chennai, incident markers, the 200 m cluster radius and a heatmap of incident density.

FastAPI: FastAPI is a modern Python web framework built on asynchronous I/O and type hints. It provides the REST API, request validation through Pydantic, automatic OpenAPI documentation and the WebSocket endpoint used for live updates.

PostgreSQL, PostGIS and SQLAlchemy: PostgreSQL stores incidents, clusters, media records, users and responders, with PostGIS available for spatial queries. SQLAlchemy 2.0 is used as an asynchronous ORM, which also allows the same code to run on SQLite in standalone mode.

Celery and Redis: Celery runs the AI pipeline as background tasks so the API can respond to the citizen immediately. Redis acts as the message broker between the API and the workers.

YOLOv8: The Ultralytics YOLOv8 nano model detects 80 classes of common objects, including people, cars, buses, trucks and motorcycles, in images and video keyframes.

Whisper: OpenAI's Whisper model transcribes voice notes recorded in the browser into text.

llama.cpp and Qwen3.5: llama.cpp runs quantised large language models efficiently on consumer hardware and exposes an OpenAI-compatible HTTP API. The system sends each report to a Qwen3.5 model with a strict JSON schema, so the model's output is guaranteed to be valid structured data.

Sentence-Transformers: The all-MiniLM-L6-v2 model converts each description into a 384-dimensional vector used for semantic duplicate detection.

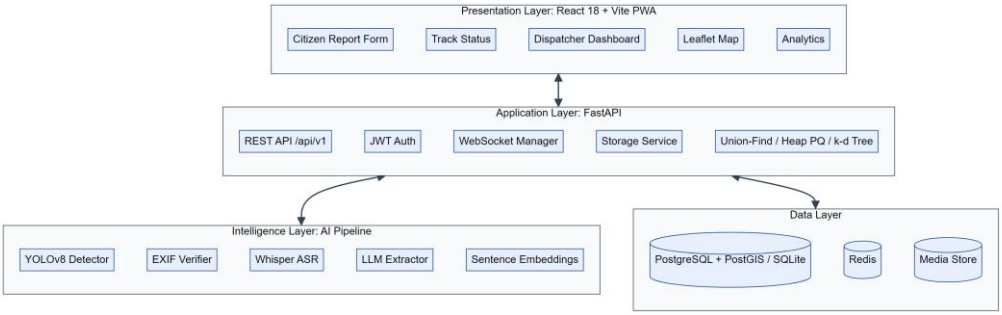


Fig 5.1 Sentinel System Components

CHAPTER 6

SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE

The system follows a four-tier architecture. The client tier contains the citizen progressive web app and the dispatcher dashboard. The application tier is a FastAPI server that exposes the REST API with JWT authentication, a WebSocket manager for live events, a k-d tree service for nearest-responder search, and a storage service for uploaded media. The data tier holds PostgreSQL, the Redis broker and the media store. The AI processing tier is a Celery worker (or an in-process background task in standalone mode) that runs each incident through vision, EXIF verification, transcription, LLM extraction, embedding, Union-Find de-duplication and priority scoring, before marking it VERIFIED and notifying the dashboard.

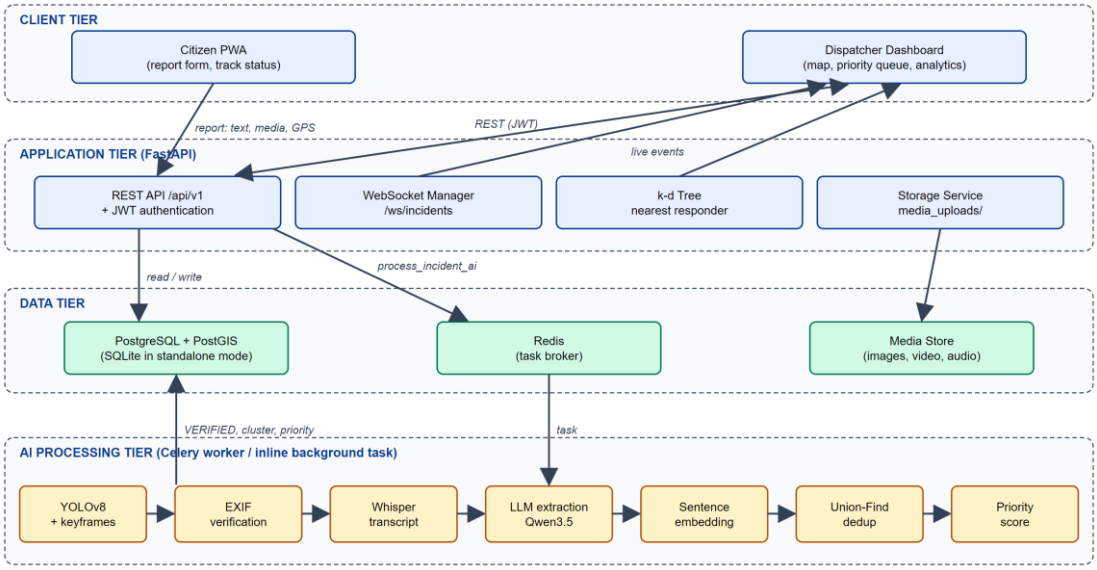


Fig 6.1 System Architecture

6.2 DATA FLOW DIAGRAM

The data flow diagram shows how information moves between external entities, processes and data stores. The citizen submits incident details, media and GPS to process 1.0, which stores the incident record and media and returns a tracking ID. Process 2.0 analyses the stored media, process 3.0 extracts structured details and an embedding from the description and the media results, and process 4.0 compares the incident with recent

verified incidents to cluster duplicates and compute the priority. Process 5.0 presents the priority queue to the dispatcher together with responder locations and records assignments, while process 6.0 returns status updates to the citizen.

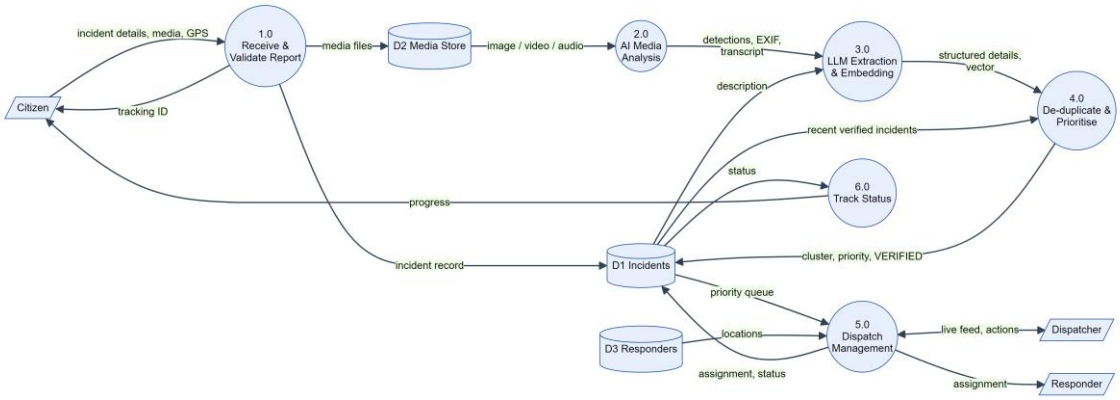


Fig 6.2 Data Flow Diagram

6.3 UML DIAGRAM

Unified Modeling Language diagrams describe the system from different viewpoints. The use case diagram shows what each actor can do, the class diagram shows the main data entities and algorithm classes, and the activity diagram shows the end-to-end flow of a report through the system.

6.3.1 Use Case Diagram

The system has four actors. The Citizen submits reports, attaches media, shares location and tracks status. The Dispatcher logs in, monitors the live map and queue, reviews AI-extracted details, finds nearest responders, dispatches them and updates the incident status. The Admin can log in and view analytics. The AI Pipeline is a system actor that analyses media and text and performs duplicate detection and prioritisation; submitting a report includes this analysis.

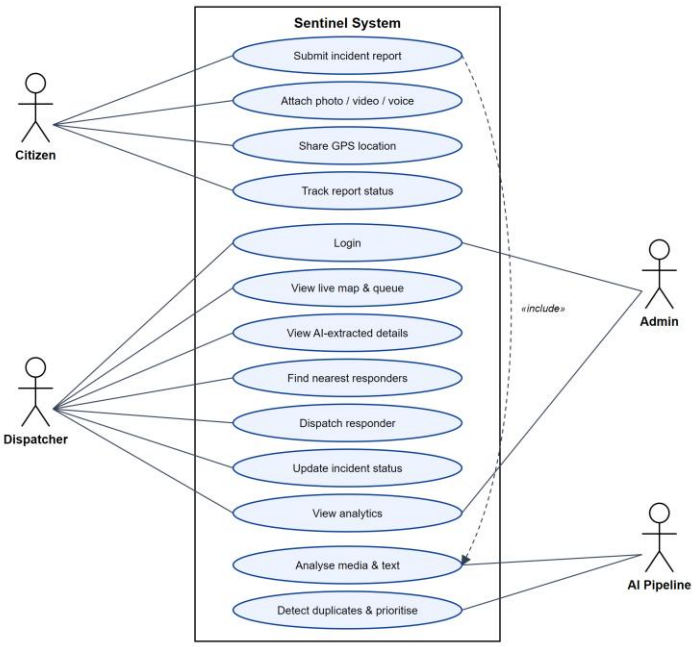


Fig 6.3.1 Use Case Diagram

6.3.2 Class Diagram

The central class is Incident, which holds the category, description, coordinates, severity, priority score, status, AI-extracted structured details and the embedding. An Incident owns zero or more MediaAttachment records and may belong to an IncidentCluster that groups duplicate reports. A Responder is linked to a User account and can be assigned many incidents. The AIPipeline class updates incidents and uses the UnionFind class for clustering, while the KDTree class indexes responder locations.

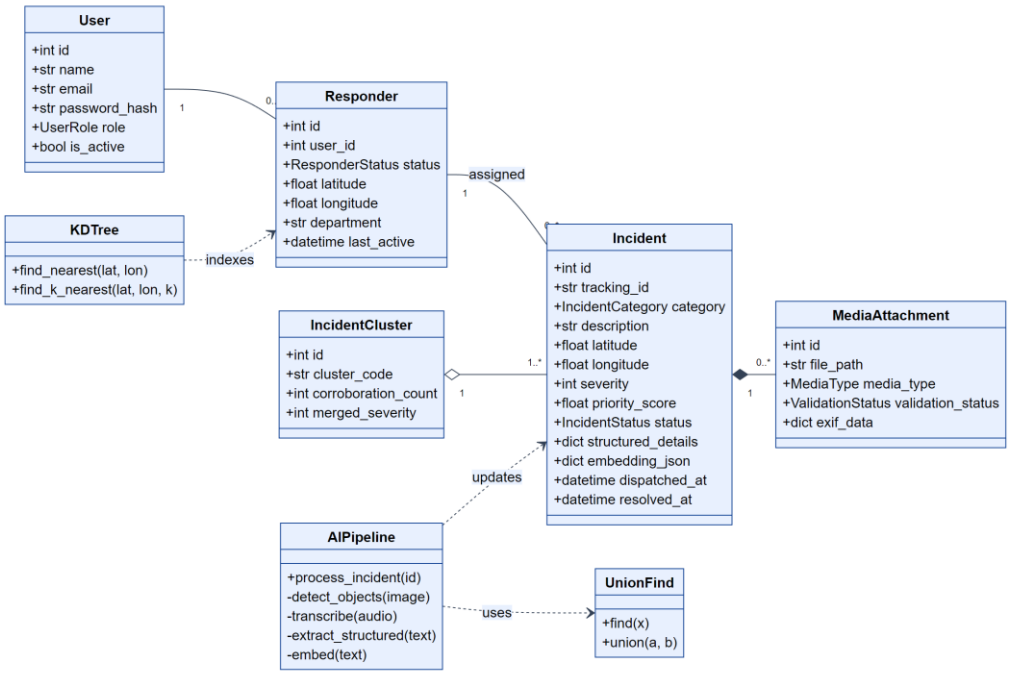


Fig 6.3.2 Class Diagram

6.3.3 Activity Diagram

The activity diagram traces a report from the moment a citizen opens the app, through validation, storage, WebSocket broadcast and AI analysis, the duplicate decision, priority scoring, dispatcher review, nearest-responder selection and dispatch, until the incident is resolved and the citizen sees the final status.

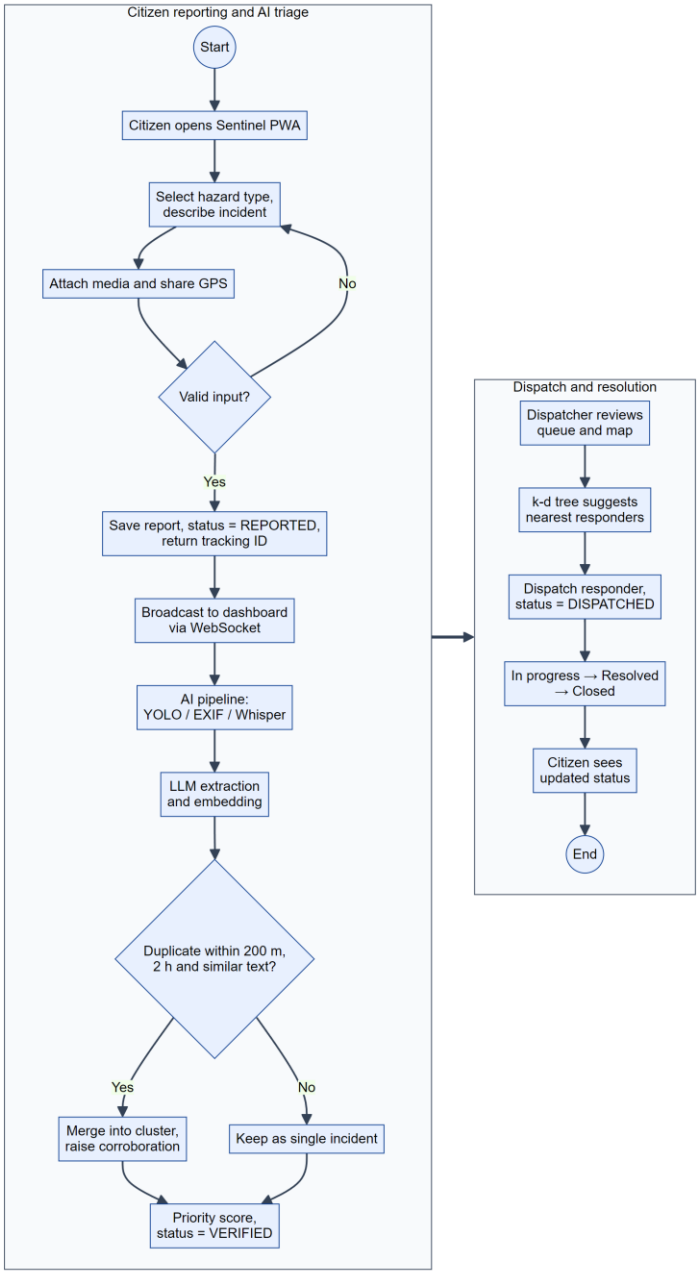


Fig 6.3.3 Activity Diagram

CHAPTER 7

SYSTEM IMPLEMENTATION

7.1 MODULES

- Citizen Reporting Module
- Media Storage and Validation Module
- Computer Vision Module
- Speech and Language Processing Module
- Duplicate Detection Module
- Priority Scoring Module
- Dispatcher Dashboard and Real-time Module
- Responder Dispatch and Authentication Module

7.2 MODULE DESCRIPTION

7.2.1 Citizen Reporting Module

- **Description**: Provides the public-facing progressive web app through which citizens report hazards and track their reports.
- Functions:
  - Category selection from eight hazard types such as Road Accident, Fire Hazard, Flooding and Medical Emergency.
  - Free-text description, camera capture or file upload of photos and videos, and in-browser voice recording.
  - GPS capture through the browser Geolocation API, with manual latitude/longitude entry as a fallback.
  - Submission as multipart form data to POST /api/v1/incidents/, returning a tracking ID.
  - Track Status screen showing the seven-stage lifecycle of a report.
- **Output**: A stored incident in status REPORTED and a tracking ID shown to the citizen.

7.2.2 Media Storage and Validation Module

- **Description**: Saves uploaded evidence safely and verifies image metadata.

- Functions:
  - Stores files under unique random names in the media store and records type and size.
  - Reads EXIF GPS and timestamp from photographs and compares them with the submitted location and time using the Haversine formula.
  - Marks each attachment as VALID or SUSPICIOUS (or reports that no EXIF data exists), without rejecting the report.
- **Output**: MediaAttachment records with validation status and EXIF details.

7.2.3 Computer Vision Module

- **Description**: Detects objects in images and videos to provide independent evidence of the incident.
- Algorithms Used:
  - YOLOv8n object detection on every image attachment.
  - **Video keyframe sampling**: three evenly spaced frames are extracted, each is run through YOLO, and the best frame is saved as a keyframe image shown to the dispatcher.
  - **Severity heuristic**: buses, trucks and fire-related objects raise the estimated severity.
- **Output**: Detected object labels, highest confidence and an estimated severity.

7.2.4 Speech and Language Processing Module

- **Description**: Converts voice notes to text and extracts structured information from every report.
- Functions:
  - Whisper transcribes audio attachments and appends the transcript to the description.
  - The LLM extractor sends the description and vision results to a local Qwen3.5 model through llama-server with a JSON schema enforced by grammar.
  - **Extracted fields**: hazard type, location name, time, affected people, severity rating, whether an immediate response is required, and a summary.
  - An ai_provenance record notes which models actually ran for each incident.
- **Output**: The structured_details JSON shown in the dispatcher's incident panel.

7.2.5 Duplicate Detection Module

- **Description**: Merges multiple reports of the same event into a single cluster.
- **Algorithm Used**: Union-Find (disjoint set union) with path compression and union by rank.
- Functions:
  - Each description is embedded with all-MiniLM-L6-v2 into a 384-dimensional vector.
  - Two incidents are duplicates if they are within 200 metres, within two hours and have a cosine similarity of at least 0.55.
  - Matching pairs are united, and each resulting set becomes an IncidentCluster with a corroboration count and merged severity.
  - Clustering is refused when only fallback (non-semantic) vectors are available.
- **Output**: Cluster codes and corroboration counts shown on the dashboard.

7.2.6 Priority Scoring Module

- **Description**: Orders incidents so the most urgent appear first.
- **Formula**: Priority = 0.40 × Severity + 0.30 × Corroboration + 0.20 × Time in queue + 0.10 × Category weight, with each factor normalised to 0–100.
- **Data Structure**: A binary heap priority queue returns the highest-priority incident in O(log n) time.
- **Output**: A priority score for every incident and a sorted activity feed.

7.2.7 Dispatcher Dashboard and Real-time Module

- **Description**: Gives dispatchers a live operational picture of the city.
- Functions:
  - Summary cards for active incidents by severity, unit status, duplicate clusters and connection status.
  - Interactive Leaflet map of Chennai with markers, 200 m cluster radius, heatmap and category filters.
  - Real-time activity feed sorted by priority, with search and category filters.
  - WebSocket connection that receives incident_created and incident_ai_processed events, with automatic reconnection and a 15-second polling fallback.

    - Analytics page showing category and status distribution, average time to dispatch and to resolve.
- **Output**: A continuously updated dashboard and analytics view.

7.2.8 Responder Dispatch and Authentication Module

- **Description**: Secures dispatcher functions and assigns responders.
- Functions:
    - Login with bcrypt-hashed passwords and HS256 JSON Web Tokens; every dispatcher endpoint requires the dispatcher or admin role.
    - A k-d tree built over available responders returns the nearest units with distances in kilometres.
    - One-click dispatch assigns the responder, records dispatched_at and moves the incident to DISPATCHED.
    - Status transitions through In Progress, Resolved and Closed are time-stamped for response-time analytics.
- **Output**: Assigned responders and complete status history for every incident.

CHAPTER 8

SAMPLE CODE

The following excerpts are taken directly from the project source code. They show the citizen report API, the AI processing pipeline, the vision detector, the duplicate detection and priority algorithms and the k-d tree used for responder search.

incidents.py – Citizen report endpoint

```
@router.post("/", response_model=IncidentRead, status_code=status.HTTP_201_CREATED)
async def create_incident(
    background_tasks: BackgroundTasks,
    category: IncidentCategory = Form(...),
    description: str = Form(...),
    latitude: float = Form(...),
    longitude: float = Form(...),
    location_name: Optional[str] = Form(None),
    phone_number: Optional[str] = Form(None),
    turnstile_token: Optional[str] = Form(None),
    files: List[UploadFile] = File(default=[]),
    db: AsyncSession = Depends(get_db)
):
    # Verify bot protection token
    is_valid_token = await verify_turnstile_token(turnstile_token)
    if not is_valid_token:
        raise HTTPException(status_code=403, detail="Anti-bot security verification
failed.")

    tracking_id = f"INC-{uuid.uuid4().hex[:8].upper()}"

    incident = Incident(
        tracking_id=tracking_id,
        category=category,
        description=description,
        latitude=latitude,
        longitude=longitude,
        location_name=location_name,
        phone_number=phone_number,
        severity=2,  # Default baseline prior to AI processing
        priority_score=50.0,
        status=IncidentStatus.REPORTED
    )
    db.add(incident)
    await db.flush()

    for upload_file in files:
        if upload_file.filename:
            file_url, file_size = await storage_service.save_file(upload_file)
            mtype = MediaType.IMAGE
            if upload_file.content_type and "video" in upload_file.content_type:
                mtype = MediaType.VIDEO
            elif upload_file.content_type and "audio" in upload_file.content_type:
                mtype = MediaType.AUDIO

            media_attachment = MediaAttachment(
                incident_id=incident.id,
                file_path=file_url,
                media_type=mtype,
                file_size_bytes=file_size,
                validation_status=ValidationStatus.PENDING
            )
            db.add(media_attachment)
```

```
    await db.commit()

    # Fire-and-forget AI processing (vision, extraction, embeddings).
```

ai_tasks.py – AI processing pipeline

```
    # A real transcript is genuine reported content, so it joins the text used
    # for extraction and embedding. An absent one contributes nothing.
    analysis_text = incident.description
    if transcripts:
        analysis_text = " ".join([incident.description] + transcripts)

    # 2. LLM Extraction. The provider always returns `model_available` and
    # `parsed`; when a real model parsed cleanly and vision had no opinion, its
    # severity_rating is the best signal we have and replaces the reporter's
    # self-assessment. Vision (a direct look at the evidence) still wins.
    extracted_details = await llm_extractor.extract_details(analysis_text)
    incident.structured_details = extracted_details
    llm_ran = bool(extracted_details.get("model_available"))
    if llm_ran and extracted_details.get("parsed") and not
provenance["vision"].startswith("yolov8"):
        rating = extracted_details.get("severity_rating")
        if isinstance(rating, int) and 1 <= rating <= 5:
            incident.severity = rating
            provenance["severity_source"] = "llm"

    # 3. Vector Embedding. Provenance travels with the vector: dedup must know
    # whether this came from a real model or the offline hash fallback.
    embedding = embeddings_engine.generate_embedding(analysis_text)
    incident.embedding_json = {
        "vector": embedding.vector,
        "model": embedding.model,
        "is_semantic": embedding.is_semantic,
    }

    # 4. Deduplication - find similar incidents and cluster them
    await _run_deduplication(incident, session)

    # 5. Priority score. Previously this only ran for incidents that landed in a
    # cluster of 2+, so every single-report incident kept the placeholder 50.0
    # it was created with and the "priority sorted" queue was sorting a constant.
    if incident.cluster_id is None:
        incident.priority_score = _recalculate_priority(incident, 1)

    # Record which components genuinely ran. structured_details is already
    # exposed on IncidentRead, so this needs no schema change and lets the UI
    # distinguish model-backed triage from a mocked pass.
    provenance["llm"] = "llm" if llm_ran else "unavailable"
    provenance["llm_backend"] =
f"{llm_extractor.provider.name}:{llm_extractor.provider.model_name}"
    provenance["embedding"] = embedding.model
    if isinstance(incident.structured_details, dict):
        incident.structured_details = {**incident.structured_details,
"ai_provenance": provenance}

    # Status VERIFIED means "AI triage complete" - _run_deduplication filters
    # candidates on it, so the transition must happen even when models are
    # mocked. Honesty about what actually ran is recorded separately.
    incident.status = IncidentStatus.VERIFIED
    await session.commit()
```

detector.py – YOLOv8 hazard detector

```
    def detect(self, image_path: str) -> Dict[str, Any]:
        self._load_model()
```

```
        if self.model is None or not os.path.exists(image_path):
            # No model or no readable image. Report that plainly instead of
            # emitting a severity and confidence the caller cannot distinguish
            # from a real detection.
            logger.warning("Vision model unavailable - no detection performed for
%s", image_path)
            return {
                "model_available": False,
                "detected_objects": [],
                "highest_confidence": None,
                "estimated_severity": None,
                "hazard_label": None,
            }

        results = self.model(image_path)
        detected_objects = []
        max_conf = 0.0

        for r in results:
            for box in r.boxes:
                label = self.model.names[int(box.cls[0])]
                conf = float(box.conf[0])
                detected_objects.append(label)
                if conf > max_conf:
                    max_conf = conf

        severity = 2
        if any(w in detected_objects for w in ["fire", "smoke", "truck", "bus"]):
            severity = 4
        elif any(w in detected_objects for w in ["car", "motorcycle", "person"]):
            severity = 3

        return {
            "model_available": True,
            "detected_objects": list(set(detected_objects)),
            "highest_confidence": round(max_conf, 2),
            "estimated_severity": severity,
            "hazard_label": detected_objects[0] if detected_objects else "Unknown",
        }
```

union_find.py – Union-Find with path compression and union by rank

```
    """
    Disjoint Set Union (DSU) data structure with path compression and union by rank.
    Used for merging duplicate reports referring to the same incident event.
    """
    def __init__(self, elements: List[int]):
        self.parent = {el: el for el in elements}
        self.rank = {el: 0 for el in elements}


    def find(self, i: int) -> int:
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i])  # Path compression
        return self.parent[i]


    def union(self, i: int, j: int) -> bool:
        root_i = self.find(i)
        root_j = self.find(j)

        if root_i == root_j:
            return False

        # Union by rank
        if self.rank[root_i] < self.rank[root_j]:
            self.parent[root_i] = root_j
        elif self.rank[root_i] > self.rank[root_j]:
            self.parent[root_j] = root_i
```

```
        else:
            self.parent[root_j] = root_i
            self.rank[root_i] += 1
        return True

    def get_clusters(self) -> Dict[int, List[int]]:
```

priority_queue.py – Weighted priority score

```
def calculate_priority_score(severity: int, corroboration_count: int,
time_in_queue_hours: float, category: str) -> float:
    """
    Weighted composite priority score formula:
    priority = severity(40%) + corroboration_count(30%) + time_in_queue(20%) +
category_weight(10%)
    """
    norm_severity = (min(max(severity, 1), 5) / 5.0) * 100.0
    norm_corroboration = (min(corroboration_count, 10) / 10.0) * 100.0
    norm_time = (min(time_in_queue_hours, 24.0) / 24.0) * 100.0
    cat_weight = CATEGORY_WEIGHTS.get(category.lower(), 5.0) * 10.0


    score = (
        norm_severity * 0.40 +
        norm_corroboration * 0.30 +
        norm_time * 0.20 +
        cat_weight * 0.10
    )
    return round(score, 2)
```

kd_tree.py – k-d tree construction

```
    def _build_tree(self, points: List[Dict[str, Any]], depth: int) ->
Optional[KDNode]:
        if not points:
            return None


        axis = depth % 2
        key_func = (lambda p: p["latitude"]) if axis == 0 else (lambda p:
p["longitude"])
        sorted_points = sorted(points, key=key_func)
        median_idx = len(sorted_points) // 2
        median_point = sorted_points[median_idx]


        node = KDNode(
            point=(median_point["latitude"], median_point["longitude"]),
            data=median_point,
            axis=axis
        )
        node.left = self._build_tree(sorted_points[:median_idx], depth + 1)
        node.right = self._build_tree(sorted_points[median_idx + 1:], depth + 1)
        return node
```

CHAPTER 9

SYSTEM TESTING AND MAINTENANCE

Testing is the process of identifying errors and ensuring that the system functions correctly according to the specified requirements. It verifies that all modules work properly and produce the expected outputs. The project contains 111 automated tests: 64 backend and AI pipeline tests written with pytest, 45 frontend unit tests written with Vitest and React Testing Library, and 2 end-to-end browser tests written with Playwright. All of them pass on the final build.

TYPES OF TESTS

9.1 UNIT TESTING

Unit testing checks individual functions and classes in isolation. The Haversine function is tested with the known distance between Chennai Central and Egmore stations (about 1.9 km), and the EXIF verifier with the distance from Chennai Central to Tambaram (about 26 km). The Union-Find structure is tested for correct merging, path compression and cluster extraction. The duplicate rule is tested with pairs that are close and similar (merged), close but semantically different (not merged) and pairs that carry only fallback vectors (never merged). The priority function is tested so that higher severity and more corroboration always increase the score, and the k-d tree is tested against a brute-force search to confirm that it returns the true nearest responders. On the frontend, each React component, the API service and the WebSocket client are tested separately.

9.2 FUNCTIONAL TESTING

Functional testing verifies that each feature meets its requirement from the user's point of view. Test cases cover submitting a report with and without media, rejecting a report without coordinates, returning a unique tracking ID, retrieving a report by tracking ID, listing and filtering incidents, updating status through the lifecycle, finding nearest responders and dispatching one. For example, when a citizen submits a report with a photograph, the API must return HTTP 201 with status REPORTED, the photo must be saved and linked as a media attachment, and after processing the incident must be VERIFIED with an ai_provenance record listing the models that ran. Functional testing

also verifies the analytics summary, which must report correct counts by category and status and the average time to dispatch and resolve.

9.3 INTEGRATION TESTING

Integration testing checks that modules work correctly together. The backend tests run the real FastAPI application against an in-memory SQLite database, so the API, ORM models, storage service and pipeline are exercised together without external services. Pipeline tests create incidents, run the full _process_incident function and check that media routing, LLM extraction, embedding, clustering and priority scoring combine correctly; for example, two reports one metre apart with matching descriptions must end up in the same IncidentCluster with a corroboration count of two. The Playwright end-to-end test starts the real frontend in a browser, fills in the citizen form, submits it and confirms that the request reaches the API with the expected fields.

9.4 BLACK BOX TESTING

Black box testing examines the system only through its inputs and outputs, without knowledge of the internal code. The citizen form was tested with valid reports, missing descriptions, missing or invalid coordinates, very long text, large images, videos and voice notes, and the dispatcher dashboard was tested by logging in with correct and incorrect credentials and attempting to access protected pages without a token. The full demonstration dataset of 18 incidents across Chennai, including three deliberate duplicate groups at T. Nagar, Kathipara junction and Velachery, was pushed through the real pipeline with the local language model; all three groups were merged into clusters and every incident reached the VERIFIED state with realistic severities and summaries.

9.5 WHITE BOX TESTING

White box testing is based on knowledge of the internal logic, and aims to exercise every important branch. Tests were written for each branch of the media router (image, video, audio), for the fallback paths taken when YOLO, Whisper, the language model or the embedding model are unavailable, for the retry without response_format when the language model server rejects a JSON schema, and for parsing model output that contains code fences or extra text. The authentication dependency is tested so that requests without a token, with an expired token or with a non-dispatcher role are rejected with the correct status codes.

CHAPTER 10

SCREENSHOTS

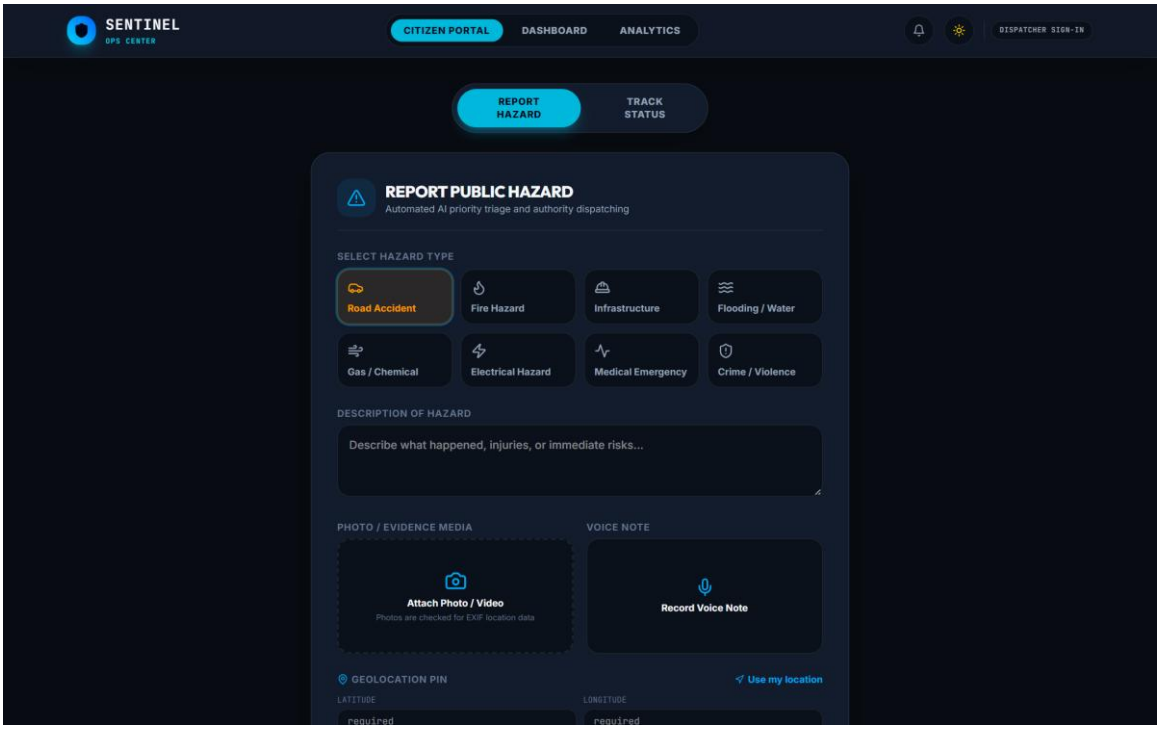


Fig 10.1 Citizen Report Form

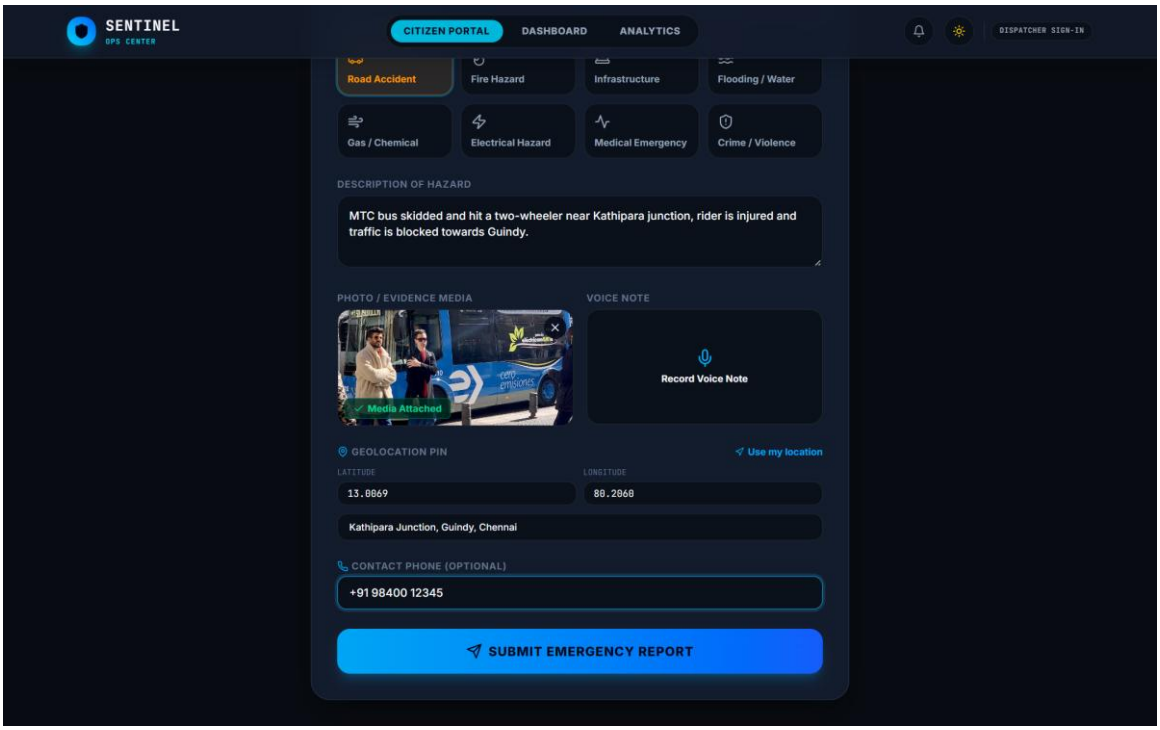


Fig 10.2 Report with Photo Evidence and GPS

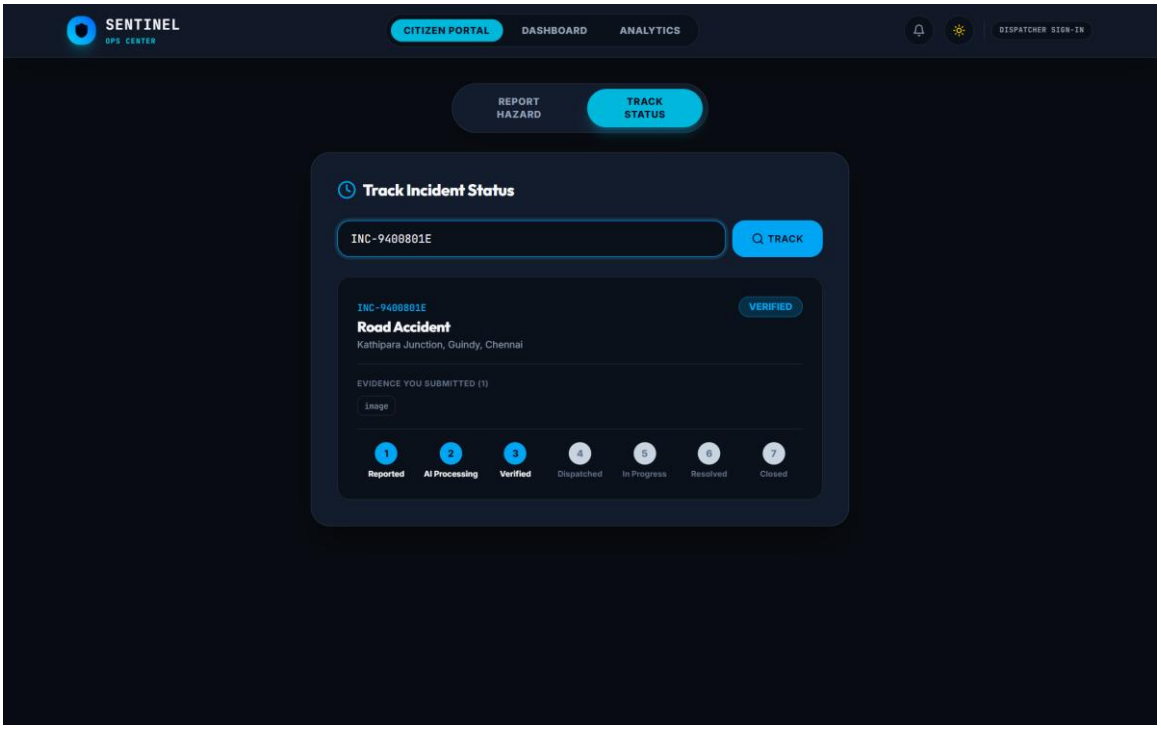


Fig 10.3 Citizen Track Status

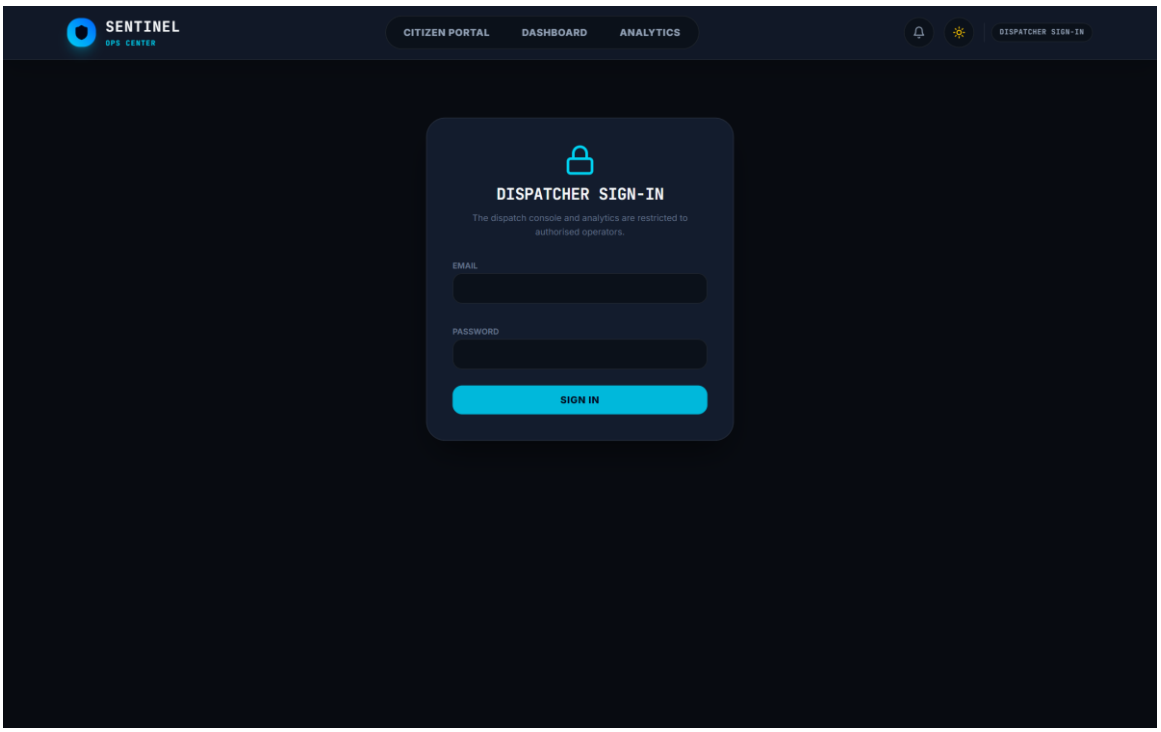


Fig 10.4 Dispatcher Login

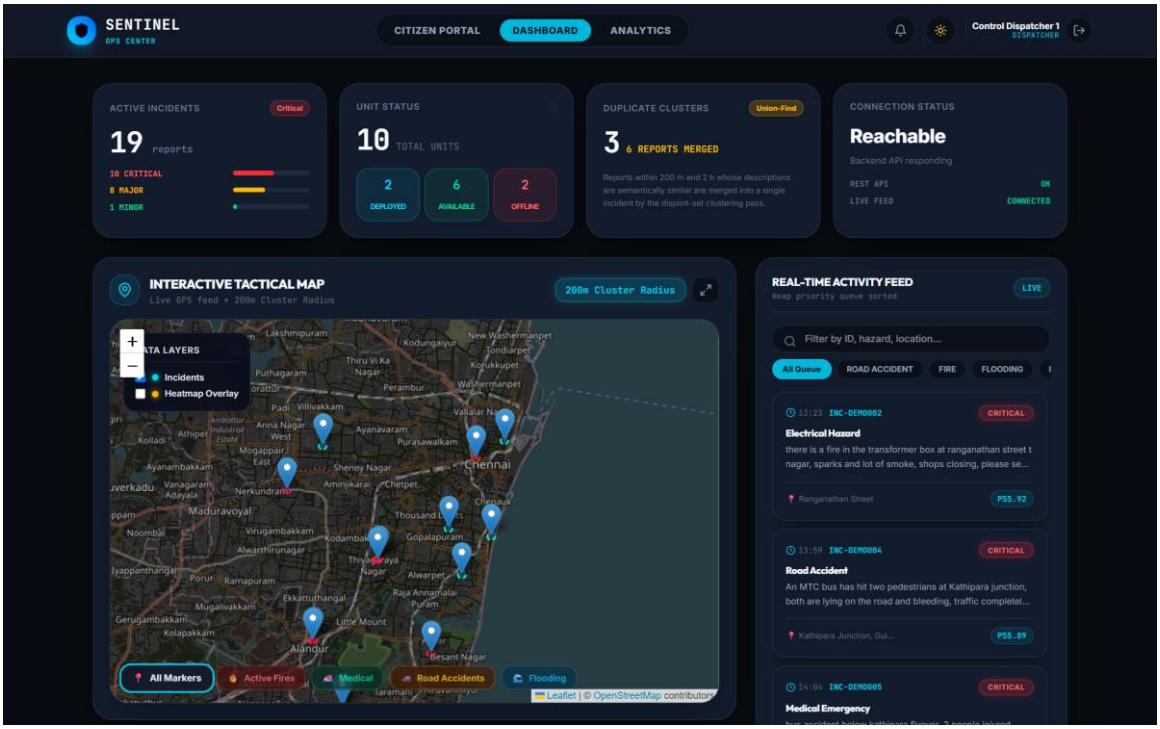


Fig 10.5 Dispatcher Dashboard

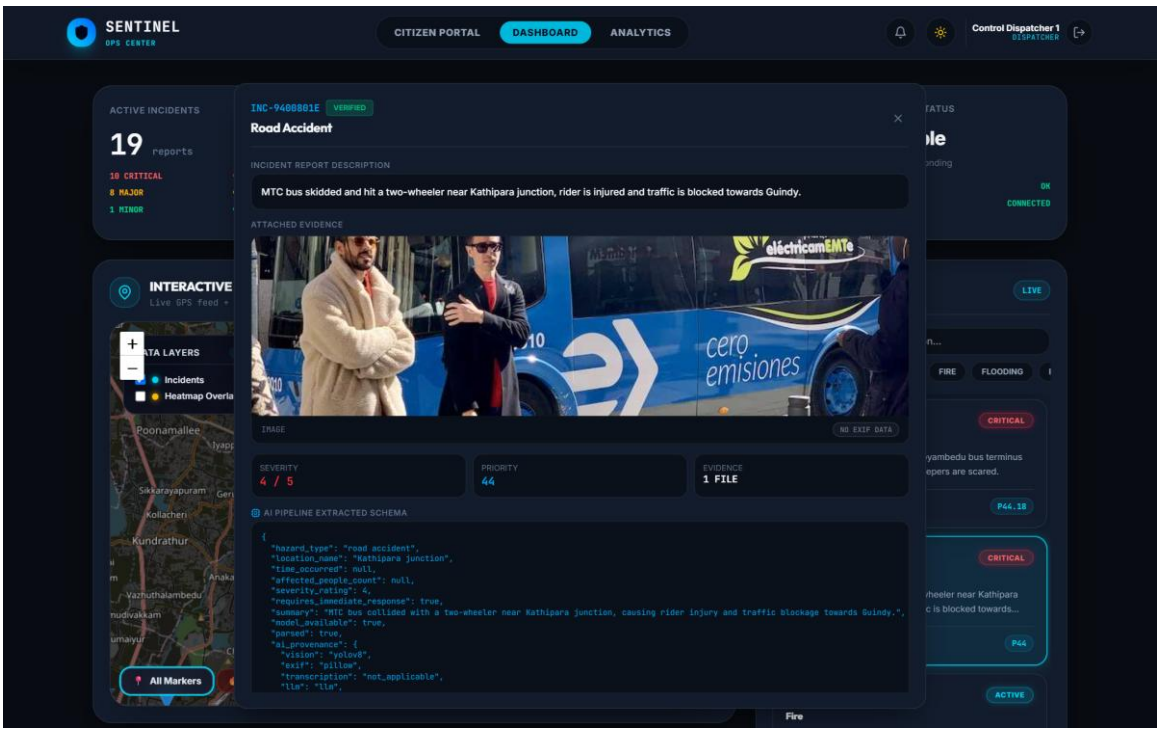


Fig 10.6 AI Incident Analysis

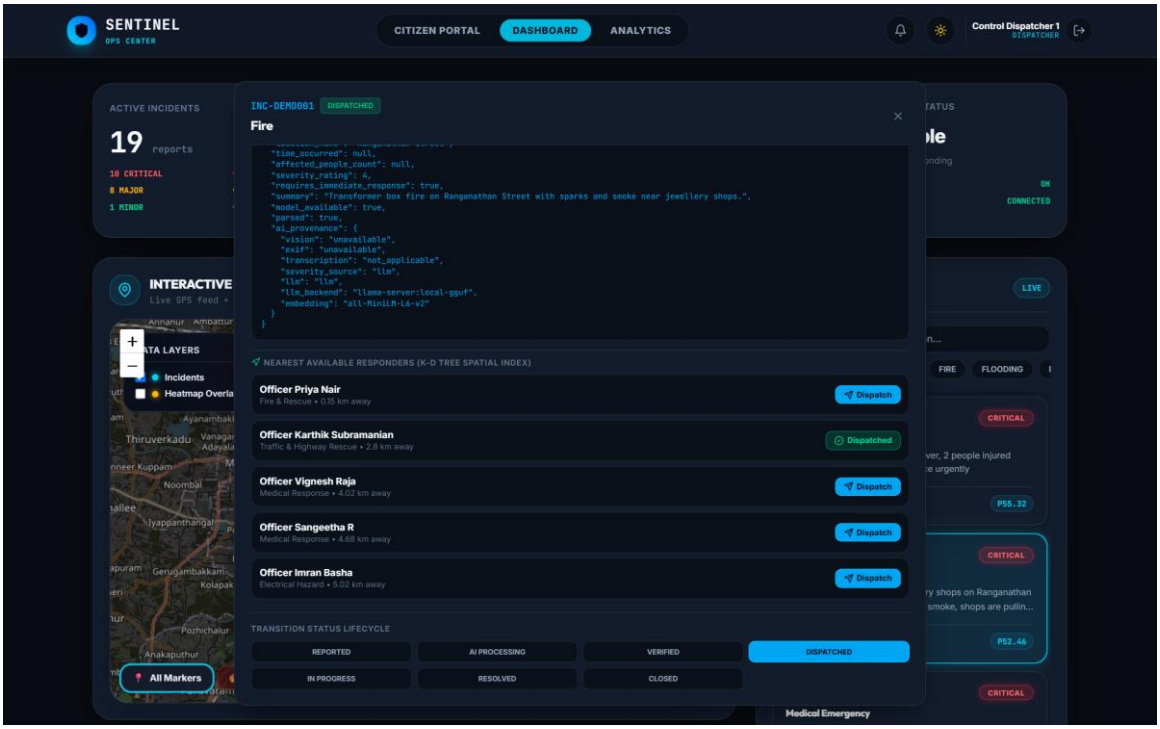


Fig 10.7 Nearest Responder Dispatch

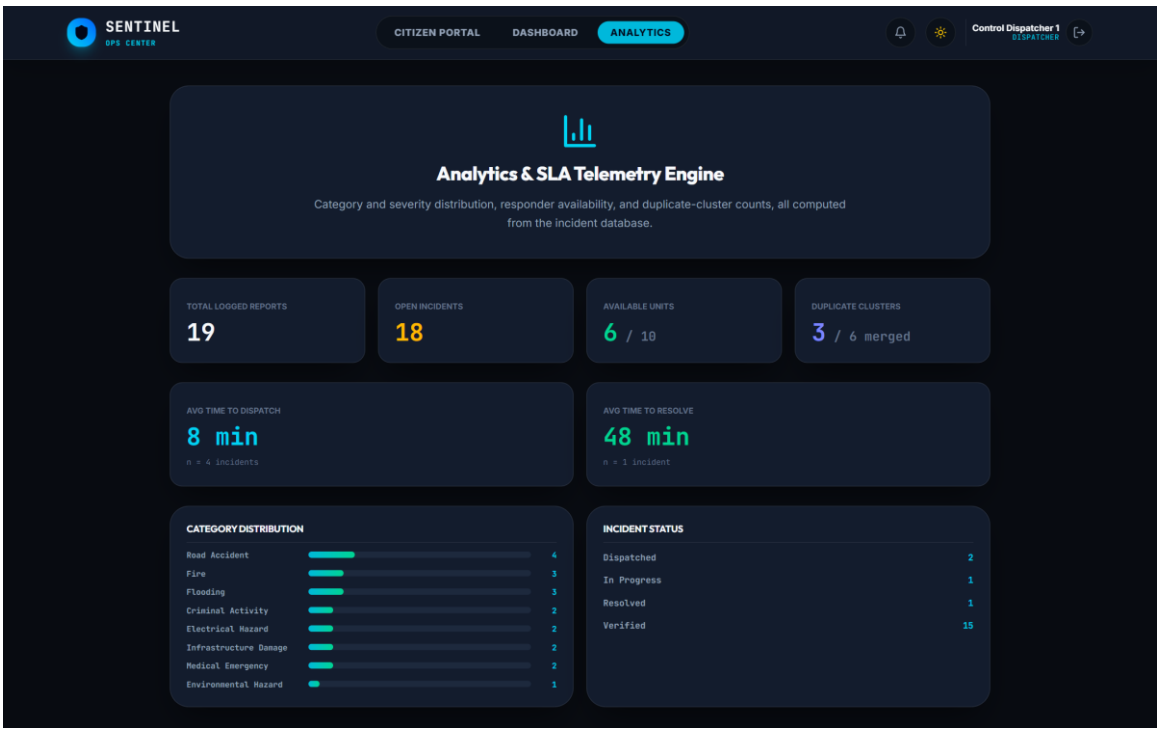


Fig 10.8 Analytics Dashboard

CHAPTER 11

CONCLUSION & FUTURE ENHANCEMENT

11.1 CONCLUSION

The Sentinel system demonstrates that artificial intelligence can turn citizen reports from a stream of unstructured, duplicated phone calls into a verified and prioritised incident queue. A citizen in Chennai can report a hazard in under a minute with a photograph, video or voice note and an exact GPS position, and receives a tracking ID immediately. Behind the scenes, YOLOv8 analyses images and video keyframes, EXIF metadata is checked against the reported location, Whisper transcribes voice notes, and a locally hosted language model extracts a structured summary and severity. Sentence embeddings with a Union-Find algorithm merged all three duplicate groups in the Chennai demonstration dataset into single clusters, and a weighted priority score placed the most serious incidents, such as the bus accident at Kathipara junction and the transformer fire at T. Nagar, at the top of the queue.

For dispatchers, the live map, real-time feed and k-d tree recommendation of the nearest available responder reduce the time spent understanding an incident and deciding whom to send. Every stage of the lifecycle is time-stamped, so the average time to dispatch and to resolve can be measured and improved. The system was designed to be honest about its own limits: whenever a model is not available, it records that fact instead of inventing results. Built entirely from open-source software and running on a single laptop with a consumer GPU, Sentinel is an affordable and practical step towards faster, smarter emergency response.

11.2 FUTURE ENHANCEMENT

- **Integration with Emergency Services** – Connect Sentinel with the Emergency Response Support System (112) and state police, fire and ambulance control rooms so that verified incidents flow directly into their dispatch systems.
- **Regional Language Support** – Add Tamil and other Indian languages to the interface and use multilingual speech recognition and language models so that every citizen can report in their own language.

- **Responder Mobile App** – Provide responders with a mobile app that shares live location, receives assignments with navigation, and updates incident status from the field.
- **Fine-tuned Hazard Detection** – Train the vision model on Indian incident images (fires, floods, road accidents, fallen wires) so that hazards themselves, not just objects, are detected.
- **Predictive Analytics** – Use historical incident data to forecast hotspots, such as flood-prone roads during the monsoon, and pre-position responders.
- **Spam and Abuse Protection** – Enable bot protection, rate limiting and reputation scoring to filter false or malicious reports at scale.
- **Offline Reporting** – Allow the progressive web app to queue reports when the network is unavailable and send them automatically when connectivity returns, including SMS fallback.
- **Scalable Cloud Deployment** – Deploy on Kubernetes with multiple Celery workers and GPU inference servers to support city-wide and state-wide use.

REFERENCES

[1] Sakaki, T., Okazaki, M., & Matsuo, Y. (2010). Earthquake Shakes Twitter Users: Real-time Event Detection by Social Sensors. Proceedings of the 19th International Conference on World Wide Web (WWW), 851–860.

[2] Imran, M., Castillo, C., Diaz, F., & Vieweg, S. (2015). Processing Social Media Messages in Mass Emergency: A Survey. ACM Computing Surveys, 47(4), Article 67.

[3] Goodchild, M. F., & Glennon, J. A. (2010). Crowdsourcing Geographic Information for Disaster Response: A Research Frontier. International Journal of Digital Earth, 3(3), 231–241.

[4] Alam, F., Ofli, F., & Imran, M. (2018). CrisisMMD: Multimodal Twitter Datasets from Natural Disasters. Proceedings of the International AAAI Conference on Web and Social Media (ICWSM).

[5] Ofli, F., Alam, F., & Imran, M. (2020). Analysis of Social Media Data using Multimodal Deep Learning for Disaster Response. Proceedings of the 17th ISCRAM Conference.

[6] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.

[7] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. Proceedings of the 40th International Conference on Machine Learning (ICML).

[8] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP 2019, 3982–3992.

[9] Bentley, J. L. (1975). Multidimensional Binary Search Trees Used for Associative Searching. Communications of the ACM, 18(9), 509–517.

[10] Aringhieri, R., Bruni, M. E., Khodaparasti, S., & van Essen, J. T. (2017). Emergency Medical Services and Beyond: Addressing New Challenges through a Wide Literature Review. Computers & Operations Research, 78, 349–368.

[11] Tarjan, R. E. (1975). Efficiency of a Good But Not Linear Set Union Algorithm. Journal of the ACM, 22(2), 215–225.

[12] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[13] Gerganov, G. et al. llama.cpp: LLM Inference in C/C++. https://github.com/ggml-org/llama.cpp

[14] Ministry of Electronics and Information Technology, Government of India. (2023). The Digital Personal Data Protection Act, 2023.